

Wordglass

wordglass.app

A scripted-video recording tool for creators and teams.
Android first, native Kotlin. Teleprompter capture, AI script generation, auto-captions, and smart clipping in a single V1.

PLATFORM

Android / Kotlin
iOS in Swift, later release

CLIENT STACK

CameraX · Media3
Compose · GLSL · ML Kit

BACKEND

Supabase · R2
AssemblyAI · Anthropic API

MONETISATION

Freemium + seats
Play Billing · Stripe

ESTIMATE

22–27 weeks
Solo build, single V1

BASELINE

targetSdk 36
minSdk 26 · Billing Lib 8

Document Structure

01	Product & Brand	<u>3</u>
	What Wordglass is, the moat, and domain status	
02	Platform Decision	<u>5</u>
	Native Kotlin, and the frameworks rejected	
03	Android Stack	<u>5</u>
	Client libraries and the media pipeline	
04	Backend Stack	<u>8</u>
	Supabase, storage, ASR, and LLM	
05	Billing Architecture	<u>9</u>
	Play fee structure and provider-agnostic entitlement	
06	Compliance Baseline	<u>11</u>
	Play deadlines that set project defaults	
07	Excluded Technologies	<u>11</u>
	What was ruled out and why	
08	iOS Port Map	<u>13</u>
	Framework equivalents for the later release	
09	Development Environment	<u>14</u>
	IDE, device loop, and distribution	
10	Build Order & Timeline	<u>16</u>
	Dependency chain and estimates	
11	Open Items	<u>17</u>
	Outstanding checks before commit	

Product & Brand

What the product does, why it can hold a position in a commodity category, and the naming decision.

1.1 The product

A script is written or AI-generated, read aloud from a teleprompter overlay while the camera records, then automatically captioned and cut into short clips. Android first. Solo creators and teams.

1.2 The wedge

No general-purpose editor owns the *recording* moment for scripted talking-head video. Every competitor works on footage that already exists. Wordglass has the script the user read, word-level timestamps, and the intended structure — ground truth nothing else in the category has access to. Smart clipping is built on that advantage; competitors are guessing at hooks from a transcript alone.

1.3 Moat mechanisms

Mechanism 01

Watermark distribution loop

Free-tier exports carry a subtle corner watermark, sized to survive platform crops on 9:16. The audience publishes to public feeds, so every export is an acquisition event rather than a private file.

Mechanism 02

Team scripts

A marketing lead writes and assigns scripts; reps record them. The billable unit becomes the org rather than the individual, so a rep leaving does not cancel the subscription.

THIRD MECHANISM — ACCRUES WITH USE

The org brand kit is injected as context into AI script generation, and approved scripts become tone samples for future generations. A team's output gets progressively more on-brand the longer they stay. This is a switching cost built from accumulated data, and it is the one thing a competitor cannot copy by shipping a feature.

1.4 Name and domain

Wordglass. The teleprompter beam-splitter is literally a piece of glass with words on it — the name describes the mechanism without being generic. Nine letters, unambiguous spelling from hearing it, no collision with billiards ("cue"), pharmaceuticals ("-ura" endings), or cleaning services ("clean-").

Research across web, Crunchbase, Play Store, and App Store surfaced **no existing commercial use** of the string. Candidates killed during the search included Cuedeck, Verbate, Scriva, Narrata, Promptu, Elocura, and Cleantake — each either live in an adjacent category or holding the obvious domain.

PRIMARY	<code>wordglass.app</code> — Google Registry TLD, HSTS-preloaded, self-describing for a mobile product
.COM STATUS	Registered and listed for sale at a high asking price. Not an obstacle: no competing business occupies it
DEFENSIVE	Register an obvious near-miss <code>.com</code> variant as a redirect target
LATER	Make an offer on the <code>.com</code> once past a few thousand in MRR — by then it is a rounding error

.APP IMPLICATIONS TO DESIGN AROUND

The entire `.app` TLD is HSTS-preloaded, so HTTPS is enforced at browser level with no fallback — good in production, mildly awkward in local development against that hostname. Some older mail filters score new gTLDs harder, so SPF, DKIM, and DMARC must be correct from the first transactional email. Verify the registry price before planning around it: Google Registry prices a portion of the `.app` namespace at premium tiers, and premium rates usually apply at every renewal rather than year one only.

Platform Decision

Native Kotlin, and the reasoning that rules out the cross-platform alternatives.

2.1 Native Kotlin, not React Native or Flutter

Count what this app does natively: CameraX preview, a GL shader injected into the capture pipeline, Media3 Transformer composition, custom shader effects at export, ML Kit face contours driving both the beauty mask and auto-reframe tracking, a frame-accurate scrub timeline, and MediaStore export. That is not a portion of the app — it is the app. A JS or Dart layer would own only script editing, auth screens, and settings.

Built in React Native, all of the above becomes native modules anyway, plus a bridge, plus serialisation cost on frame-level state. That is the worst of both models.

2.2 Why "but iOS later" does not change this

The composition layer does not port regardless of framework choice. Media3 Transformer and AVFoundation share no API surface, and the shaders need Metal rather than GLSL. What genuinely ports is the backend, the schema, the prompts, and the design system — none of which depend on the mobile framework.

2.3 Kotlin Multiplatform: rejected

There is less to share than it appears. The media pipeline does not share, and it is the bulk of the app. Auth, networking, and entitlement are thin wrappers over Supabase, which has SDKs for both platforms. The genuinely interesting shared logic — segment extraction, filler-word detection, script generation, entitlement resolution — belongs on the server anyway, because it already depends on the LLM and the ASR result.

PRINCIPLE

The shared layer is the backend, not a mobile framework. Design the API so the iOS client is a thin consumer of it, and the port becomes a UI-plus-media job rather than a rewrite.

Android Stack

Client libraries, with the media pipeline detailed separately since it carries the technical risk.

CONCERN	CHOICE	NOTES
Language / UI	Kotlin + Jetpack Compose	Compose Preview drives most UI iteration without a device

CONCERN	CHOICE	NOTES
minSdk	26 (Android 8.0)	Dropping to API 21 adds only ~3.7 points of reach and pulls in 2014-era devices
targetSdk	36 (Android 16)	Play requirement from 31 Aug 2026 — see Section 06
Camera	CameraX 1.5+	<code>CameraEffect</code> injects an OpenGL <code>SurfaceProcessor</code> into PREVIEW, VIDEO_CAPTURE and IMAGE_CAPTURE. The <code>camera-effects</code> artifact ships OpenGL implementations, reducing boilerplate
Composition & export	Media3 Transformer 1.10+	Hardware-accelerated via MediaCodec, OpenGL for graphical modification. Compatible from API 23
Editor preview	<code>CompositionPlayer</code>	Compositor settings apply to the <code>Composition</code> and work with both <code>CompositionPlayer</code> for preview and <code>Transformer</code> for export — one object graph, no second rendering path
Face detection	ML Kit face contours	Use the <i>thin</i> variant backed by Play Services: ~20.3 MB smaller than bundled. Contours need faces at 200×200 px or larger
Async	Coroutines + Flow	
Background export	WorkManager + foreground service	Persistent notification, backgroundable, completion notification
DI	Hilt	
Local persistence	Room	Projects, takes, transcript state. Local is the source of truth; cloud is a sync target
Networking	Ktor client or Retrofit	Either is fine — pick one and stop thinking about it
Crash / perf	Sentry	Covers iOS later; better Kotlin ergonomics than Crashlytics
Product analytics	PostHog	Track which features are used before a paid conversion

3.1 Media pipeline detail

All Media3 APIs below are confirmed present in the Transformer demo and release notes as of 1.10 (March 2026).

API	USED FOR
<code>Composition</code> , <code>EditedMediaItemSequence</code>	Trimming, multi-take stitching, sequence assembly
<code>OverlayEffect</code> , <code>BitmapOverlay</code>	Captions and logo. Captions drawn to Bitmap via Canvas, fed as time-ranged overlays
<code>SingleColorLut</code>	Colour grade presets — built in, no custom shader work needed
<code>GLEffect</code> / custom <code>GLShaderProgram</code>	Skin smoothing, warmth, brightness lift at export
<code>AudioProcessor</code>	Voice / music-bed mixing and ducking
<code>SilenceSkippingAudioProcessor</code>	Partial head start on silence trimming
<code>Crop</code> + ML Kit tracking	Auto-reframe landscape → 9:16, low-pass-filtered pan

3.2 Shader architecture

Write the GLSL once and use the same source in two places: as a `CameraEffect` for live preview, and as a Media3 `GLEffect` at export. **Record clean, bake at export.** Effects are therefore non-destructive and changeable after the take is shot, so a bad filter choice never costs a take.

HARD PERFORMANCE BUDGET

For 30fps input, each frame must be processed in under 30ms or frames drop. That is the ceiling for the beauty shader on the slowest device you intend to support. ML Kit Selfie Segmentation measures 25–65ms depending on input size — at the top of that range it exceeds the live-preview budget entirely, which is why background replacement is deferred and would run at export rather than in preview.

BEAUTY SCOPE

Skin smoothing, warmth, brightness lift, and LUT presets. All off by default, user-adjusted only. No face reshaping, jaw slimming, or eye enlargement — mesh warping is the expensive part of the pipeline, ML Kit Face Mesh is beta with no SLA, and the effect is wrong for an audience that wants to look rested rather than altered.

Backend Stack

Deliberately thin. One async round-trip per video does not justify an orchestrator.

CONCERN	CHOICE	NOTES
Database	Supabase Postgres	RLS policies on <code>org_id</code>
Auth	Supabase Auth	Email + Google sign-in
Server logic	Edge Functions (Deno)	LLM calls, signed upload URLs, ASR job submission, ASR callback webhook
Video / audio storage	Cloudflare R2	\$0.015/GB-month, zero egress. Class A writes \$4.50/M, Class B reads \$0.36/M
Brand assets	Supabase Storage	Auth-scoped small files stay where RLS handles authorisation in one place
ASR	AssemblyAI	~\$0.15/hr batch. Word-level timestamps native
ASR fallback	Deepgram Nova-3	~\$0.26/hr batch. Its advantages are streaming latency and per-second billing — neither needed here
LLM	Anthropic API	One integration, two prompts: script generation and segment extraction
Push	FCM	Script assignment notifications
Web dashboard	Next.js on Vercel	Writer surface — see 4.2

4.1 What is deliberately absent

No Railway, no Inngest, no FastAPI service. A continuous multi-source pipeline needs a job orchestrator; this app's async work is one webhook round-trip per video. Adding an orchestrator now buys nothing and costs a service to operate.

No Firestore or Firebase Realtime Database either. Relational queries across orgs, scripts, assignments, and usage counters are the core access pattern, and RLS enforces tenancy at the database rather than in application code. Firestore security rules are a worse fit for a role-based org model.

4.2 The web dashboard is not optional

The writer persona — a marketing lead producing ten scripts a week — will not do that on a phone. The team-scripts moat depends on a surface the mobile app does not provide. Scope is small: script library, brand kit editor, assignment, review inbox. Budget 3–4 weeks inside the V1 estimate.

DAY-ONE ARCHITECTURAL REQUIREMENT

Org-scoped data model with `org_id` on every row, from the first migration. A solo user is simply an org of one. Retrofitting multi-tenancy after several hundred solo accounts is a migration to avoid rather than survive.

4.3 Unit economics

ASR is the constraint to price around: **captions are metered, the teleprompter is not**. Script generation is cheap per call but will be hammered, so it is metered separately rather than folded into the same allowance. Two independently metered resources give two soft paywall triggers alongside the watermark, so conversion does not depend on a single moment of friction.

SECTION 05

Billing Architecture

The Google Play fee structure changed materially on 30 June 2026. The design below reflects the current rules.

5.1 Current Play service fees

Following the Epic injunction, Google no longer requires the use of Google Play Billing for apps serving US users, and separated the service fee from the billing fee.

PATH	REGION	SUBSCRIPTION COST
Google Play Billing	US / UK / EEA	10% + 5% = 15%
Alternative billing or external web link	US / UK / EEA	10% + processor ≈ 13%
Google Play Billing	Rest of world, incl. Kenya	15%

The 5% billing fee applies only when the purchase completes through Google Play Billing. The subscription service fee is flat at 10% in those three regions regardless of install status or revenue level — there is no \$1M graduation cliff on subscriptions.

5.2 Decision

SOLO SUBSCRIPTIONS	Google Play Billing, in-app. The ~2 point saving from routing to a web checkout does not justify the conversion drop-off, nor the PCI-DSS, dispute-handling and customer-support obligations that come with alternative billing
TEAM SEATS	Stripe on the web dashboard. Web-originated B2B revenue sits outside Play's scope entirely
PRICING	Monthly and annual only. No weekly plan — Play scrutinises weekly-plus-trial patterns and it is the single biggest driver of one-star reviews in this category

PROVIDER-AGNOSTIC ENTITLEMENT – BUILD ON DAY ONE

A single `subscription` table that both Play and Stripe write into, with the app querying only that table and never a payment provider directly. The reason is optionality rather than fee arbitrage: this area is still moving. Google entered a new settlement with Epic in March 2026 with a revised Modified Injunction pending, and gave notice in July 2026 that developers in the US alternative-billing and external-links programs must report transactions and pay service fees from 1 October 2026.

5.3 Free tier

FREE • UNLIMITED	Teleprompter recording, local takes, trim, beauty effects, audio import
FREE • METERED	Caption minutes (start at 10/month) and AI script generations
FREE • WATERMARK	Subtle corner mark on every export, positioned to survive platform crops on 9:16
PAID	Removes the watermark, raises caption minutes and generation cap

Compliance Baseline

Play requirements that set project defaults from the first commit rather than being handled later.

REQUIREMENT	DEADLINE	DETAIL
targetSdk 36 (Android 16)	31 Aug 2026	New apps and updates must target API 36 to be submitted. Extension available to 1 Nov 2026
Play Billing Library 8+	31 Aug 2026	All new apps and updates. Extension available to 1 Nov 2026
Existing-app floor	31 Aug 2026	Apps must target API 35 or higher to stay available to new users on newer devices

BEHAVIOUR CHANGE THAT TOUCHES THIS APP

Apps targeting Android 16 can no longer opt out of edge-to-edge mode. Content extends behind the status and navigation bars unless explicitly handled — directly relevant to a full-bleed camera preview with a scrolling text overlay. Handle insets deliberately in the capture UI rather than discovering this during a compliance scramble.

Start the project on `targetSdk 36` and Billing Library 8. Neither is a migration if it is the starting point.

Excluded Technologies

Ruled out with reasoning recorded, so the decisions are not relitigated mid-build.

TECHNOLOGY	REASON
FFmpegKit	Retired 6 Jan 2025; binaries pulled from Maven Central, CocoaPods and npm on 1 Apr 2025. Cited reasons were the effort of tracking upstream FFmpeg and codec patent exposure following MPEG LA's acquisition by Via-LA. A maintained continuation exists (FFmpegKitNext), but Media3 covers the need with hardware acceleration and no patent question
On-device Whisper	Too slow on the mid-range Android hardware this audience actually owns
Third-party beauty SDKs	DeepAR and Banuba run several thousand per year — unjustified when the shader work is 2–3 weeks
React Native / Flutter / KMP	See Section 02

Firestore / Firebase RTDB	Wrong fit for a relational, role-based org model
Inngest / Railway / FastAPI	One webhook per video does not need an orchestrator
AI eye-contact correction	Research-grade real-time gaze redirection. Ships on desktop GPUs. Not viable on a Snapdragon 6-series
One-click Instagram posting	No public content-publishing API for personal accounts. Every competitor's version is the share sheet
One-click YouTube posting	Data API upload works but needs OAuth verification review and costs 1,600 quota units per upload against a 10,000 daily default. Deferred
Bundled music library	Licensing liability. Users import their own audio via SAF
Chroma key	ML Kit Selfie Segmentation covers the same job without a green screen — but it is beta and outside the live-preview frame budget. Deferred, not dead

iOS Port Map

Swift and SwiftUI. Framework equivalents for the later release, recorded now so the Android boundaries are drawn correctly.

ANDROID	IOS
Kotlin + Jetpack Compose	Swift + SwiftUI
CameraX	AVFoundation — <code>AVCaptureSession</code> , <code>AVCaptureVideoDataOutput</code>
<code>CameraEffect</code> + GLSL	Metal shaders, or Core Image filters
Media3 Transformer	<code>AVMutableComposition</code> + <code>AVVideoComposition</code> + <code>AVAssetExportSession</code>
<code>OverlayEffect</code> / <code>BitmapOverlay</code>	<code>CALayer</code> via <code>AVVideoCompositionCoreAnimationTool</code>
<code>SingleColorLut</code>	<code>CIColorCube</code> / <code>CIColorCubeWithColorSpace</code>
<code>AudioProcessor</code>	<code>AVAudioMix</code> with <code>MTAudioProcessingTap</code>
Room	GRDB or SwiftData
WorkManager + foreground service	<code>BGProcessingTask</code> — see caveat below
MediaStore	PhotoKit (<code>PHPhotoLibrary</code>)
FCM	APNs
ML Kit face contours	ML Kit iOS, or Apple's Vision framework

Two areas get easier on iOS. Core Image supplies skin smoothing, warmth and LUTs largely as configured filters rather than hand-written shaders, so the beauty work shrinks. Vision's face landmark detection is first-party and needs no third-party SDK.

THE ONE GENUINELY HARDER AREA

Background export. WorkManager plus a foreground service lets an Android export run as long as it needs with a persistent notification. iOS grants a few minutes of background execution with no equivalent guarantee. The export flow must either complete in-foreground or checkpoint and resume — a real UX difference, not a porting detail.

Budget iOS at 60–70% of the Android effort, not 20%. What carries over is the product decisions, schema, prompts, design system, and every bug already found. To keep the port a matter of writing a second implementation rather than untangling the first, keep the media layer behind clean interfaces — `CaptureEngine`, `ExportPipeline`, `EffectStack` — and never let CameraX or Media3 types leak into view models.

SECTION 09

Development Environment

IDE choice, the device loop, and how builds reach testers.

9.1 IDE

Android Studio for the app. VS Code for everything else — Edge Functions, the Next.js dashboard, migrations.

JetBrains shipped an official Kotlin extension for VS Code in Alpha at KotlinConf 2026, powered by the Kotlin Language Server. However, its build-system support lists Gradle and Maven with *experimental* Android Gradle Plugin support, and JetBrains states plainly that IntelliJ IDEA and Android Studio provide the best Kotlin experience. More decisively: Live Edit, Compose Preview, Layout Inspector, and the GPU and memory profilers are Android Studio features. On an app whose core risk is a 30ms frame budget and hardware-encoder export times, the profiler is not optional.

If the keybindings are the attachment: `Settings → Keymap → VSCode`.

9.2 Claude Code integration

The Claude Code plugin officially supports Android Studio alongside IntelliJ IDEA, PyCharm, WebStorm, PhpStorm and GoLand. Unlike the VS Code extension it does not bundle the CLI, so Node and the `claude` CLI install separately, then the plugin from the marketplace, then a full IDE restart.

- Quick launch with `Cmd+Esc` / `Ctrl+Esc`
- Changes open in the IDE diff viewer rather than the terminal
- Current selection or tab is shared automatically
- `Cmd+Option+K` / `Alt+Ctrl+K` inserts file references such as `@File#L1-99`
- **IDE diagnostics — lint and syntax errors — are shared with Claude automatically as you work.** Gradle sync failures and Kotlin compiler errors land in context without copy-pasting stack traces
- `/ide` from any external terminal connects an existing CLI session to the running IDE

The SDD loop is unchanged — specify, plan, tasks, implement uncommitted, review, commit — but with `/ide` connected the implement step's changes surface in Android Studio's diff viewer for the review pass.

9.3 Device loop

Pair wireless ADB once and leave it. After the initial USB pairing, Run deploys over WiFi.

```
# Phone: Developer options → Wireless debugging → Pair device with pairing code
adb pair 192.168.1.42:37891      # enter the 6-digit code
adb connect 192.168.1.42:5555
```

Keep the dev build installable alongside any release build:

```
buildTypes {
    debug {
        applicationIdSuffix = ".dev"
        versionNameSuffix = "-dev"
    }
}
```

NOT AN EXPO-STYLE LIVE-UPDATE BUILD

Expo installs a dev client once, which then fetches JS bundles from a dev server over the network — the phone does not care where the editor is. Kotlin compiles to bytecode, and Live Edit works by Android Studio pushing recompiled classes into the already-running app over ADB. The phone must be connected to the dev machine, Android Studio must be running the session, and only changes inside a composable's body apply live; new functions, changed signatures and new classes force a rebuild. **There is no install-once, updates-forever artifact.**

9.4 Distribution to testers

CHANNEL	USE
Firebase App Distribution	Free, upload from Gradle or CI, testers get an install link. No Play review. Best for early testers and design reviewers
Play internal app sharing	Upload an APK or AAB for an immediate shareable link, no review. Needed to test the real Play install path and anything Billing-related
Play internal testing track	Up to 100 testers, signed AAB, closest to production. Required for subscription testing

THE EMULATOR WILL NOT SERVE THIS PROJECT

Camera passthrough is a webcam feed, GL shader performance on a host GPU is meaningless, and hardware encoder behaviour during export is precisely what needs validating. Budget two or three real phones early — one recent flagship-class device and one mid-range Snapdragon, since the mid-range is where the beauty shader and export times will actually break.

Build Order & Timeline

The dependency chain is not negotiable even though the release is a single V1.

- 1

Camera, teleprompter, takes
CameraX preview with scrolling overlay, take storage and picker
- 2

Export pipeline
Media3 Transformer — the GL pipeline steps 5 and 6 both depend on
- 3

Captions
AssemblyAI integration producing word-level timestamps
- 4

Transcript editor
Correction UI; the surface every automated cut is reviewed in
- 5

Smart clipping
Depends entirely on 3 and 4. Filler removal, then segments, then auto-reframe
- 6

Beauty shaders
Depends on the GL pipeline from step 2
- 7

Auth and orgs
Org-scoped schema was in place from the first migration; this turns on the UI
- 8

AI script generation
A generic version can ship earlier for testers, but brand-kit context from step 7 is what makes it good
- 9

Web dashboard
Script library, brand kit, assignment, review inbox
- 10

Team review loop
Notifications, approve / request-retake, comments
- 11

Billing
Play Billing for solo, Stripe for seats, one entitlement table

10.1 Estimates

COMPONENT	ESTIMATE
Teleprompter and captions core	8–10 weeks
Smart clipping	included above
Beauty shaders	2–3 weeks
Auth, orgs, script sync	3–4 weeks

AI script generation and import	2–3 weeks
Web dashboard	3–4 weeks
Audio import	< 1 week
Total, solo build	22–27 weeks

10.2 If scope must be cut

In this order. These cut cleanest without breaking anything downstream.

- 1 Multi-take stitching**
The take picker alone is sufficient for the core workflow
- 2 Approve / request-retake review loop**
Script assignment alone still makes the org the billing unit, which is the point
- 3 Beauty filters**
Table stakes rather than a reason anyone switches apps

SECTION 11

Open Items

Outstanding checks before the name and stack are locked.

- **Confirm `wordglass.app` is unregistered** and check the registry price tier — Google Registry prices part of the `.app` namespace as premium, and premium rates recur at renewal
- **Trademark search** — USPTO and EUIPO, classes 9 and 42, for "Wordglass" before it goes on anything
- **App-store search** — exact string on Google Play and the App Store
- **Register a defensive `.com` variant** as a redirect target, and note the asking price on `wordglass.com` for a later offer
- **Benchmark ASR on real audio** before committing — AssemblyAI against Deepgram on your own recordings, since published accuracy figures are self-reported
- **Validate the 30ms frame budget** with a trivial shader on the target mid-range device before building the full beauty stack
- **Confirm Play Console policy status** for targetSdk and Billing Library requirements at the time of first submission — the deadlines above may have moved

STANDING PRINCIPLE

The build is not the hard part. Teleprompter apps are a commodity and there are dozens already on Play. **The moat is the watermark distribution loop, the team account structure, and the accumulating brand-kit context** — not the feature list.

Every scope decision in this document, and every one made after it, should be tested against whether it strengthens one of those three. A feature that strengthens none of them makes the app better and the business no more defensible.